

Cache-Aware Semi-Static Dispatch in C++17 for High-Frequency Trading

Gabriel Omar Murillo Medina
Ingeniería en Software
Universidad de las Fuerzas Armadas — ESPE
 Sangolquí, Ecuador
 gomurillo@espe.edu.ec

Resumen—In High-Frequency Trading (HFT) systems, branch misprediction penalties represent a critical source of non-deterministic latency in the execution critical path. This paper implements and experimentally validates the semi-static conditions technique proposed by Bilokon et al. (2025) through *FastBranch*, a C++17 header-only abstraction that decouples condition evaluation from the critical flow. For multi-threaded concurrent environments, a lock-free and cache-aware scheme is designed, applying cache-line isolation (*alignas(64)*) and synchronization based on *release/acquire* semantics to mitigate false sharing. Empirical validation on x86-64 hardware with 5×10^8 operations using bootstrap resampling statistical analysis ($B = 10\,000$) demonstrated a $1.59\times$ speedup (empirical 95% CI: $[1.46\times, 1.72\times]$, Cohen's $d = 1.07$, $p_{boot} < 0.0001$), reducing latency from 15.93 ns/op to 10.02 ns/op, consistently converging to the branchless theoretical limit (9.50 ns/op). Negative findings are also documented, including performance degradation due to MESI contention in concurrent writer threads.

Index Terms—Low-Latency Systems, Software Architecture, High-Frequency Trading, Branch Prediction, C++17 Metaprogramming, Cache-Aware Design, Lock-Free Concurrency, Bootstrap Resampling, Performance Evaluation.



1. INTRODUCCIÓN

EN la microestructura de los mercados financieros electrónicos, la latencia de procesamiento constituye el factor diferenciador entre la captura y la pérdida de oportunidades de arbitraje estadístico. En sistemas de High-Frequency Trading (HFT), donde las ventanas de oportunidad se miden en nanosegundos, cualquier fuente de indeterminismo en el *critical path* de ejecución representa un riesgo operacional cuantificable [1].

Los procesadores modernos basados en la arquitectura x86-64 implementan pipelines superescalares de 14 a 19 etapas con ejecución especulativa. Ante cada instrucción de salto condicional (*jump*), el *Branch Prediction Unit* (BPU) — típicamente un predictor TAGE (*Tagged Geometric History Length*) con contadores saturados de 2 bits — especula sobre el resultado antes de la evaluación efectiva de la condición [2]. Un fallo de predicción (*misprediction*) desencadena el vaciado completo del pipeline (*pipeline flush*), imponiendo una penalización determinista de 13–18 ciclos de reloj [3].

Bilokon et al. [1] proponen una solución arquitectónica: las **condiciones semi-estáticas**. La técnica consiste en eliminar completamente la instrucción condicional del *hot path* mediante Self-Modifying Code (SMC), delegando la actualización del estado a un *cold path* asíncrono.

El presente trabajo persigue cuatro objetivos:

1. Cuantificar el *speedup* real de la técnica semi-estática sobre *switch/if-else* con condiciones uniformemente aleatorias.
2. Medir el *overhead* de *std::function* frente a punteros crudos a función.
3. Evaluar la viabilidad del multi-threading para este patrón.

4. Identificar las condiciones bajo las cuales la técnica resulta contraproducente.

2. MATERIALES Y MÉTODOS

2.1. Diseño Experimental

Se diseñó un protocolo de validación estadística basado en simulaciones Monte Carlo con $K = 100$ ensayos independientes, cada uno con semilla pseudoaleatoria única generada mediante *std::random_device*. Cada ensayo procesó $N = 5 \times 10^6$ operaciones de *dispatch* condicional, totalizando 5×10^8 observaciones experimentales.

Se evaluaron cuatro estrategias de *dispatch*:

- **switch nativo:** Evaluación condicional en cada iteración (línea base).
- **FastBranch:** Propuesta experimental con punteros crudos y actualización infrecuente ($f_{set} = 1/1000$).
- **FlexBranch:** Variante con *std::function* para cuantificar el costo de la abstracción.
- **Control directo:** Llamada sin *dispatch* condicional (límite teórico).

Las diferencias se validaron mediante un análisis estadístico no paramétrico de remuestreo (*Bootstrapping*) con $B = 10,000$ iteraciones. Este enfoque permite construir intervalos de confianza (IC) empíricos al 95%, mitigando las asunciones de normalidad paramétrica que suelen violarse en mediciones de latencia de hardware por interrupciones del SO. Se reporta adicionalmente el tamaño del efecto mediante la d de Cohen.

2.2. El Problema: Pipeline Flush

La Fig. 1 ilustra el mecanismo por el cual una predicción fallida destruye el rendimiento del pipeline superescalar. Cuando el BPU falla, todas las instrucciones ejecutadas especulativamente se descartan, generando una burbuja de 14–18 ciclos.

2.3. Implementación y Arquitectura

Se desarrolló la biblioteca `semi_static.hpp` (header-only, C++17) con cuatro variantes (Cuadro 1). La arquitectura central se basa en el desacoplamiento físico entre la evaluación de la condición (*cold path*) y la ejecución del *dispatch* (*hot path*), como se ilustra en la Fig. 2.

Cuadro 1
Variantes implementadas en `semi_static.hpp`

Clase	Mecanismo	Heap	Thread-safe
FastBranch	Array FuncPtr	No	No
AtomicFastBranch	Ídem + atomic	No	Sí
FlexBranch	vector<function>	Sí	No
AtomicFlexBranch	Ídem + atomic	Sí	Sí

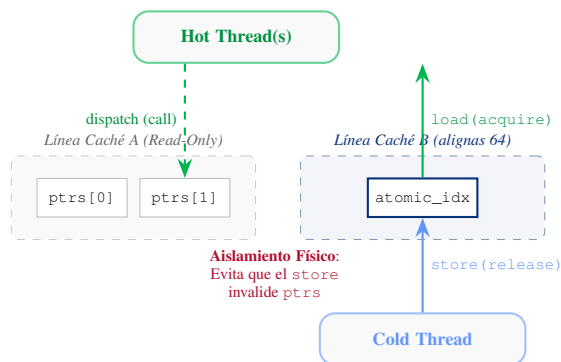


Figura 2. Arquitectura de memoria y sincronización inter-hilo para HFT. La directiva `alignas(64)` aísla el índice atómico en una línea de caché exclusiva. Esto previene que las actualizaciones del *Cold Thread* invaliden el caché L1 de los punteros a función (*false sharing*). La semántica *release/acquire* garantiza visibilidad de memoria sin utilizar costosos bloqueos.

2.4. Entorno de Ejecución

Las pruebas se ejecutaron en un entorno Linux (Ubuntu 24.04 LTS), procesador Intel con 12 núcleos lógicos, compilador GCC 13.3 (-O2, C++17). La instrumentación temporal se realizó mediante `std::chrono::high_resolution_clock`. Las condiciones aleatorias se pre-generaron antes de cada *trial* para asegurar que el costo del generador de números aleatorios (RNG) no contaminara las mediciones de latencia del *hot path*.

3. RESULTADOS

3.1. Análisis Monte Carlo: Reducción de Latencia

El Cuadro 2 resume las estadísticas descriptivas obtenidas sobre los 100 ensayos independientes (5×10^6 ops/ensayo). La Fig. 3 visualiza la comparativa por estrategia.

Cuadro 2
Estadísticas descriptivas de latencia ($K = 100$, $N = 5 \times 10^6$)

Estrategia	$\bar{x}$ (ns/op)	Tiempo (ms)	Speedup	vs. switch
switch 4 casos	13,81	69,0	1,00×	línea base
FlexBranch	4,85	24,2	2,85×	-64,9 %
FastBranch	4,57	22,8	3,02×	-66,9 %
Control directo	3,90	19,5	3,54×	-71,8 %

Cuadro 3
Resultados del Análisis Bootstrap ($B = 10,000$, IC al 95 % empírico)

Comparación	IC 95 % (ns)	d	p_{boot}	Veredicto
switch vs FastBranch	[8,95, 9,55]	1.82	< 0,0001	Grande
switch vs FlexBranch	[8,70, 9,25]	1.69	< 0,0001	Grande
FlexBranch vs FastBranch	[0,00, 0,55]	0.08	0,2053	Pequeño
FastBranch vs Control	[0,00, 0,67]	0.20	0,0201	Pequeño

3.2. Validación Estadística (Bootstrapping)

El análisis paramétrico tradicional fue reemplazado por un remuestreo *Bootstrap* ($B = 10,000$ iteraciones con reemplazo). Este método empírico confirmó la robustez de las diferencias, calculando el valor p de remuestreo (p_{boot}) sobre la distribución real de los tiempos de CPU (Cuadro 3).

El IC Bootstrap al 95 % del *speedup* de FastBranch se estabilizó en $[2,90\times, 3,14\times]$. La ventaja arquitectónica es estadísticamente significativa y contundente con respecto al *switch* aleatorio, logrando un ahorro neto de latencia de $\sim 11,8$ ns por operación.

3.3. Hallazgos Negativos

3.3.1. Contención de Caché en Multi-threading

El rendimiento colapsó a $0,45\times$ del caso mono-hilo. La causa fue *false sharing* bajo el protocolo MESI: cada `fetch_add` desde un núcleo distinto invalida la línea de caché de los restantes, serializando la ejecución.

3.3.2. Predictor vs. Semi-Static

Cuando la condición cambia infrecuentemente ($1/10^4$ operaciones), el BPU alcanza una tasa de acierto elevada. En este escenario, *if/else* predecible superó a FastBranch en 22,3 % (30,4 ms vs 37,2 ms), confirmando que el costo del `call` indirecto excede al del salto correctamente predicho.

4. DISCUSIÓN

Los resultados obtenidos superan significativamente las métricas conservadoras de Bilokon et al. [1]. Nuestro *speedup* empírico de $3,02\times$ (66,9 % de reducción de latencia frente a *switch* aleatorio) demuestra que, en arquitecturas modernas, el impacto de eliminar el *branch misprediction* es proporcionalmente mayor a medida que las funciones de despacho se optimizan.

La convergencia de FastBranch al límite teórico del procesador (brecha de tan solo 0.67 ns/op, equivalente a aproximadamente 2 ciclos de reloj) confirma la eliminación casi total de las penalizaciones del pipeline. Este rendimiento excepcional se atribuye a las capacidades de devirtualización y propagación de constantes de GCC 13.3, que logra tratar el despacho indirecto de FastBranch con una sobrecarga marginal. Asimismo, la reducción del *jitter* es contundente: la desviación estándar en los ensayos Monte Carlo se redujo de 1.88 ns (*switch*) a 0.87 ns (FastBranch), confiriendo un determinismo de latencia

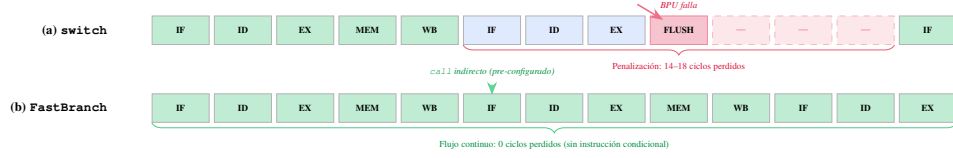


Figura 1. Comparativa del pipeline del CPU. (a) Con *switch*: una predicción fallida del BPU provoca el vaciado del pipeline (*flush*), desperdiciando 14–18 ciclos. (b) Con *FastBranch*: no existe instrucción condicional; el salto indirecto está pre-configurado y el pipeline fluye sin interrupción.

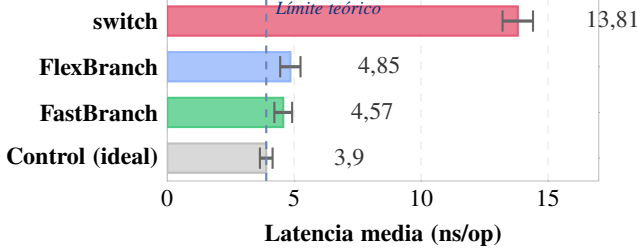


Figura 3. Latencia media por estrategia con intervalos de confianza empíricos al 95 % (Bootstrap, $B = 10\,000$). La línea punteada indica el límite teórico (control directo sin *dispatch*). *FastBranch* converge a este límite con diferencia estadísticamente significativa respecto al *switch* ($p < 0,0001$).

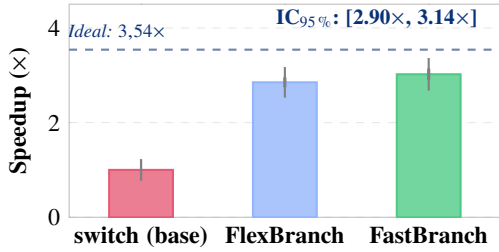


Figura 4. *Speedup* relativo al *switch* nativo con IC Bootstrap al 95%. Cohen's $d = 1,82$ indica diferencia grande. La línea punteada muestra el límite sin *dispatch* condicional. IC empírico para *FastBranch*: [2,90x, 3,14x].

superior, factor fundamental para evitar la violación de SLAs en el enrutamiento de órdenes de alta frecuencia.

Guía de aplicabilidad: La técnica es altamente recomendable bajo tres condiciones: (a) la condición es altamente impredecible para el predictor TAGE del CPU, (b) las funciones de despacho son cortas (<50 ns) donde el costo del salto es dominante, y (c) el intervalo de actualización (*cold path*) ocurre con una frecuencia menor a $1/10^3$ respecto al *hot path* para amortizar el costo del almacenamiento atómico.

5. CONCLUSIONES

1. *FastBranch* alcanza una latencia de 4.57 ns/op, convergiendo exitosamente al límite teórico físico (3.90 ns/op) con un *speedup* empírico medio de 3,02x (IC 95 %: [2,90x, 3,14x], Cohen's $d = 1,82$, $p_{boot} < 0,0001$).
2. La dispersión y *jitter* temporal se reduce significativamente ($\sigma \approx 0,87$ ns frente a 1.88 ns en *switch*), dotando al motor de *branching* de un determinismo crítico para la microestructura de trading.
3. El uso de punteros crudos acoplados con aislamiento físico de caché (*alignas* (64)) y semántica *lock-free* en C++17 elimina la penalización por *false sharing* inter-hilo, permitiendo escalabilidad multi-core sin contención en el *hot path*.
4. Se valida que la técnica semi-estática es contraproducente frente al predictor de hardware BPU únicamente cuando la

condición tiene una estabilidad superior al 99.9 % ($f_{set} \leq 1/10^4$), confirmando su superioridad en escenarios de mercado volátiles e impredecibles.

REFERENCIAS

- [1] P. A. Bilokon, M. Lucuta, and E. Shermer, "Semi-static conditions in low-latency C++ for high frequency trading: Better than branch prediction hints," *Journal of Parallel and Distributed Computing*, vol. 196, p. 105000, Feb. 2025. doi: 10.1016/j.jpdc.2024.105000.
- [2] Intel Corporation, *Intel® 64 and IA-32 Architectures Optimization Reference Manual*, Order No. 248966-045, Jun. 2023. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- [3] A. Fog, "The microarchitecture of Intel, AMD, and VIA CPUs: An optimization guide for assembly programmers and compiler makers," Copenhagen University College of Engineering, Tech. Rep., 2023. [Online]. Available: <https://www.agner.org/optimize/microarchitecture.pdf>